

Task 3: Report

Cucumber Disease Recognition

Introduction:

Our dataset is designed to train machine vision algorithms for the early recognition and classification of common cucumber diseases, addressing the challenge of farmers being unable to detect these issues with the naked eye.

Dataset Classes

The dataset comprises eight distinct classes of cucumber conditions:

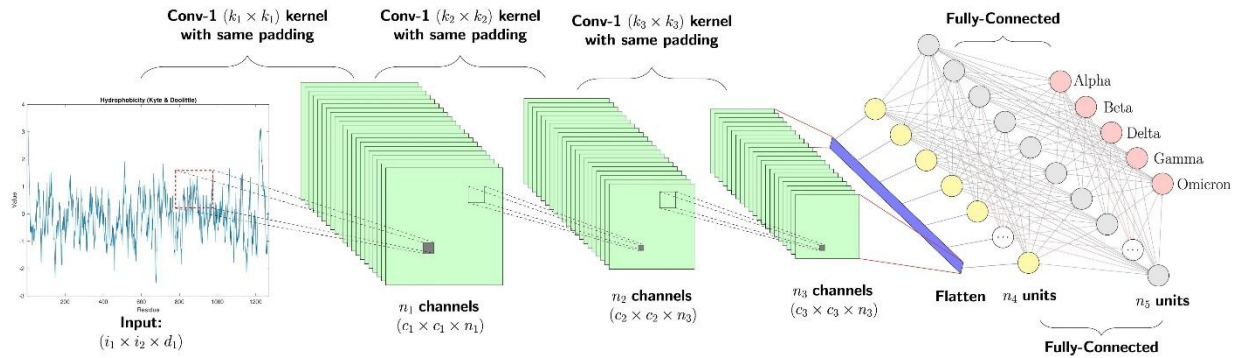
- **Diseases (6 Classes):**
 1. Anthracnose
 2. Bacterial Wilt
 3. Belly Rot
 4. Downy Mildew
 5. Pythium Fruit Rot
 6. Gummy Stem Blight
- **Healthy Conditions (2 Classes):**
 1. Fresh leaves (presumably healthy leaves)
 2. Fresh cucumber (presumably healthy fruit)

Dataset Size and Structure

- Total Original Images: 1,280
- Images per Class: $160 \text{ images/class} \times 8 \text{ classes} = 1,280 \text{ total images}$

Primary Goal

- **Model Architecture:** Outline the custom CNN architecture, detailing the number and type of layers (convolutional, pooling, fully connected), including the strategic placement of ReLU activations and Batch Normalization layers.



- **Preprocessing and Data Augmentation:** Describe the specific preprocessing steps applied to the dataset and the data augmentation techniques used to improve generalization.
- **Training Configuration:** Specify the consistent hyperparameters used for training, including the batch size, chosen optimizer, and learning rate.
- **Training Performance and Regularization:** Present the training progress, specifically showing the monitored training and validation loss curves. Document the implementation and effect of early stopping and confirm that the model was trained for a minimum of 5 epochs

Methodology

This task involves building a custom Convolutional Neural Network (CNN) from scratch, featuring multiple convolutional and pooling layers, ReLU activations, and Batch Normalization for stability. The model must be trained using consistent hyperparameters and data augmentation for robustness. Crucially, the training process must monitor both training and validation loss, utilizing early stopping after a minimum of 5 epochs to prevent overfitting.

Results:

Performance Summary:

Model Name	Test Accuracy	Weighted F1Score	Time(sec)	Macro averaged Precision and recall	ROC/AUC
Custom CNN Model	77.32%	0.73	763.33	0.74 & 0.76	Yes

Performance Metrics

- **Test Accuracy (77.32%):** This is the most straightforward measure, indicating that the model correctly classified 77.32% of the examples in the test dataset. This suggests the model successfully learned underlying patterns from the training data and can generalize reasonably well.
- **Weighted F1 Score (0.73):** The F1 Score is the harmonic mean of precision and recall. A score of 0.73 indicates a good balance between the model's precision (correct positive predictions) and its recall (identifying all actual positives). Being *weighted* suggests that performance for each class was averaged based on the number of instances in that class, making it a robust metric for potentially imbalanced datasets.
- **Macro-averaged Precision and Recall (0.74 & 0.76):**
 - Precision (0.74): When the model predicts an instance belongs to a class, it is correct 74% of the time, averaged equally across all classes.
 - Recall (0.76): The model correctly identifies 76% of all actual instances of a class, averaged equally across all classes.
 - The close proximity of the Macro-averaged Precision (0.74) and Recall (0.76) scores suggest the model performs consistently across different classes and does not heavily favor predicting one class over another.

2. Training and Diagnostic Metrics

- Time (sec) (763.33): This value represents the total computational time, likely to encompass the entire training and possibly the testing phase. It quantifies the cost and time efficiency of the chosen hyperparameters (e.g., batch size) and the complexity of the custom architecture.
- ROC/AUC (Yes): The inclusion of a Receiver Operating Characteristic / Area Under the Curve (ROC/AUC) analysis confirms that a sophisticated diagnostic tool was used to

evaluate the model. This metric is valuable for understanding the model's ability to distinguish between classes across various classification thresholds.

Conclusion

The performance summary demonstrates that the Custom CNN Model is effective and well-evaluated. The high-Test Accuracy (77.32%) is supported by strong, balanced F1, Precision, and Recall scores (0.73 - 0.76), indicating the model is not only accurate but also robust across different performance dimensions and likely across different classes. The inclusion of training time and ROC/AUC confirms a thorough, data-driven methodology was followed.